

BUG REPORT — HW04 EShop SUT

- **Sinh viên:** Ninh Văn Khải — MSSV **23127060**
- **Feature phụ trách:** FR-03 (Quên/Đặt lại mật khẩu) · FR-08 (Thanh toán) · FR-15 (Quản lý sản phẩm)
- **Môi trường:** Windows 11 · Node v22.20.0 · Playwright 1.62.1 · Chromium/Firefox/WebKit

· backend `http://localhost:3000` · web `http://localhost:5173` · admin `http://localhost:5174`

- **Nguồn bằng chứng:** ảnh trong `evidence/bugs/` do `automation/scripts/capture-bug-evidence.mjs`

chụp bằng Playwright thật; log response trong `evidence/bugs/capture-log.txt` ;

test case tương ứng trong `automation/tests/*.spec.js` (249/249 pass — xem `report/03-RUN-SUMMARY.md`).

⚠ **Cách đọc "test pass":** các test gắn mã bug được viết để khẳng định *hành vi sai hiện tại*.

Test **pass** ⇒ bug **vẫn còn**. Khi SUT được sửa, chính các test đó sẽ **fail** — đó là dấu hiệu cần cập nhật kỳ vọng.

Thang Severity: `Critical` (mất tiền / lộ dữ liệu / chiếm quyền) · `High` (sai nghiệp vụ nặng)

· `Medium` (sai nghiệp vụ nhẹ, UX xấu) · `Low` (mỹ quan, không ảnh hưởng dữ liệu)

Tổng quan

ID	Tiêu đề	Feature	Severity	Priority	Test case	Evidence
BUG-03-01	Regex mật khẩu của FE mâu thuẫn với chính thông báo lỗi của nó	FR-03	High	P1	FR03-TC04, TC05e, TC05f	BUG-03-01.png , BUG-03-01-after.png
BUG-03-02	Mã đặt lại mật khẩu bị trả về trong HTTP response và hiện thẳng trên UI	FR-03	Critical	P0	FR03-TC06	BUG-03-02.png
BUG-03-03	Mã đặt lại chỉ 4 chữ số, không giới hạn số lần thử	FR-03	Critical	P0	FR03-TC07, TC08	capture-log.txt
BUG-03-04	/api/forgot-password không có rate limit	FR-03	High	P1	FR03-TC08	capture-log.txt
BUG-03-05	Phân biệt email tồn tại / không tồn tại (user enumeration)	FR-03	Medium	P2	FR03-TC09	capture-log.txt
BUG-03-06	Backend không kiểm tra độ mạnh mật khẩu — đặt được mật khẩu 1	FR-03	High	P1	FR03-TC10, TC10a, TC10b	capture-log.txt
BUG-03-07	Mật khẩu lưu plaintext và trả về qua GET /api/users/me	FR-03	Critical	P0	FR03-TC11	capture-log.txt
BUG-03-08	Đặt lại mật khẩu thành công nhưng tài khoản vẫn bị khoá	FR-03	High	P1	FR03-TC14	BUG-03-08.png
BUG-08-01	Khách hàng tự sửa tổng tiền thanh toán ngay trên giao diện	FR-08	Critical	P0	FR08-TC04	BUG-08-01.png , BUG-08-01-success.png
BUG-08-02	/api/checkout chấp nhận tổng tiền âm và tổng tiền không phải số	FR-08	High	P1	FR08-TC05, TC06	capture-log.txt
BUG-08-03	Tạo được đơn hàng từ giỏ hàng rỗng	FR-08	Medium	P2	FR08-TC07	capture-log.txt
BUG-08-04	IDOR — ai cũng đọc được đơn hàng của người khác, không cần đăng nhập	FR-08	Critical	P0	FR08-TC08	BUG-08-04.png
BUG-08-05	Giỏ hàng không được xoá sau khi thanh toán ⇒ đặt trùng đơn	FR-08	High	P1	FR08-TC09	test log
BUG-08-06	Đơn hàng luôn được lưu với địa chỉ giao hàng rỗng	FR-08	High	P1	FR08-TC10	BUG-08-01 log
BUG-08-07	Mã giảm giá phần trăm làm TĂNG tổng tiền gấp 10 lần	FR-08	Critical	P0	FR08-TC11, TC11a, TC11b	BUG-08-07.png
BUG-08-08	Đơn hàng đúng bằng giá trị tối thiểu bị từ chối mã giảm giá (off-by-one)	FR-08	Medium	P2	FR08-TC15b	test log
BUG-08-09	Giỏ hàng biến mất khi tải lại trang	FR-08	Medium	P2	FR08-TC17	test log
BUG-15-01	Sửa 1 sản phẩm làm toàn bộ bảng đổi sang cùng một tên	FR-15	Critical	P0	FR15-TC05	BUG-15-01-before.png , BUG-15-01.png
BUG-15-02	Thêm/sửa/xoá sản phẩm không cần đăng nhập	FR-15	Critical	P0	FR15-TC07, TC08, TC09	BUG-15-02.png

ID	Tiêu đề	Feature	Severity	Priority	Test case	Evidence
BUG-15-03	Tài khoản người dùng thường tạo được sản phẩm (không phân quyền theo role)	FR-15	Critical	P0	FR15-TC10	capture-log.txt
BUG-15-04	Chấp nhận sản phẩm có tên rỗng / thiếu tên / chỉ khoảng trắng	FR-15	Medium	P2	FR15-TC11a, TC11b, TC11c	test log
BUG-15-05	Chấp nhận giá âm, giá bằng chữ	FR-15	High	P1	FR15-TC12a, TC12b, TC13	test log
BUG-15-06	Giá vượt MAX_SAFE_INTEGER bị sai lệch giá trị	FR-15	Medium	P2	FR15-TC14a	test log
BUG-15-07	Xoá sản phẩm không tồn tại vẫn báo thành công	FR-15	Medium	P2	FR15-TC15	capture-log.txt
BUG-15-08	GET /api/products/:id với id không tồn tại trả 200 {} thay vì 404	FR-15	Medium	P2	FR15-TC16	test log
BUG-15-09	Sản phẩm id chẵn trả price kiểu chuỗi, id lẻ trả kiểu số	FR-15	High	P1	FR15-TC17a, TC17b	test log
BUG-15-10	Chấp nhận category_id trở vào danh mục không tồn tại	FR-15	Medium	P2	FR15-TC18	test log
BUG-15-11	Payload XSS lưu nguyên vẹn vào CSDL, không hề được làm sạch	FR-15	High	P1	FR15-TC19	test log

Tổng: 28 bug — FR-03: 8 · FR-08: 9 · FR-15: 11.

Theo mức độ: **Critical 9** · High 10 · Medium 9.

CHI TIẾT

BUG-03-01 — Regex mật khẩu của FE mâu thuẫn với chính thông báo lỗi của nó

Feature	FR-03
Severity / Priority	High / P1
Component	frontend-web/src/pages/ForgotPassword.jsx:26
Test case	FR03-TC04 , FR03-TC05e , FR03-TC05f
Evidence	evidence/bugs/BUG-03-01.png , BUG-03-01-after.png

Precondition: tài khoản đã tồn tại, đã lấy được mã OTP hợp lệ.

Steps:

- Mở /forgot-password , nhập email, bấm **Lấy mã OTP**.
- Nhập đúng mã OTP hiển thị.

3. Nhập mật khẩu mới `NewPass123!` (12 ký tự, có hoa, thường, số, **ký tự đặc biệt**).

4. Bấm **Đặt lại mật khẩu**.

Expected: mật khẩu đủ mạnh $\Rightarrow$ đổi thành công (đúng theo thông báo mà chính FE hiển thị:

"*Phải dài tối thiểu 8 ký tự, gồm chữ hoa, chữ thường, số và KÝ TỰ ĐẶC BIỆT*").

Actual: bị chặn với alert "*Mật khẩu quá yếu!...*". Đăng nhập bằng mật khẩu đó $\Rightarrow$ 401 .

Ngược lại, `New Pass 123` (**không** có ký tự đặc biệt, chỉ có dấu cách) lại được chấp nhận.

Nguyên nhân:

```
const flawedStrongPasswordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*\s)[A-Za-z\d\s]{8,}$/;
```

- `(?=.*\s)` **bắt buộc phải có khoảng trắng** — không hề được nêu trong thông báo.
- Character class `[A-Za-z\d\s]` **cấm mọi ký tự đặc biệt** — trái ngược hoàn toàn với thông báo.

Đề xuất: `/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}$/` và bỏ ràng buộc khoảng trắng.

BUG-03-02 — Mã đặt lại mật khẩu bị trả về trong HTTP response và hiện thẳng trên UI

Feature	FR-03
Severity / Priority	Critical / P0
Component	backend/server.js:76-79 · frontend-web/src/pages/ForgotPassword.jsx:18
Test case	FR03-TC06
Evidence	evidence/bugs/BUG-03-02.png

Steps: POST `/api/forgot-password` với `{"email": "<email bất kỳ đã đăng ký>"}` (không cần đăng nhập).

Expected: mã đặt lại chỉ được gửi tới **kênh riêng của chủ tài khoản** (email/SMS); API chỉ trả thông báo chung.

Actual (log thật):

```
POST /api/forgot-password  $\Rightarrow$  200 {"message":"Mã đặt lại mật khẩu đã được tạo","resetToken":"6052"}
```

FE còn hiển thị thẳng lên màn hình: "*Mã OTP của bạn là: 6052*".

Tác động: bất kỳ ai biết email nạn nhân đều **chiếm được tài khoản** chỉ bằng 2 request, không cần truy cập hộp thư.

BUG-03-03 — Mã đặt lại chỉ 4 chữ số, không giới hạn số lần thử

Feature	FR-03 · Severity / Priority Critical / P0
Component	backend/server.js:70
Test case	FR03-TC07 , FR03-TC08

`Math.floor(1000 + Math.random() * 9000)` ⇒ không gian khoá chỉ **9000 giá trị**.

Kết hợp với BUG-03-04 (không rate limit), toàn bộ không gian có thể quét trong vài giây.

Token cũng **không có thời hạn** — `server.js:66-82` không lưu thời điểm phát hành.

Đề xuất: token ngẫu nhiên ≥ 128 bit (`crypto.randomBytes(32).toString('hex')`), có `expires_at` ≤ 15 phút, khoá sau 5 lần thử sai.

BUG-03-04 — `/api/forgot-password` không có rate limit

Feature	FR-03 · Severity / Priority High / P1 · Test case FR03-TC08
----------------	---

10 request liên tiếp cho cùng một email đều trả `200` , không có `429` , không CAPTCHA, không delay.

Mỗi lần gọi còn **ghi đè** token cũ ⇒ có thể dùng để khoá vĩnh viễn khả năng khôi phục của nạn nhân.

BUG-03-05 — User enumeration qua mã trạng thái

Feature	FR-03 · Severity / Priority Medium / P2 · Test case FR03-TC09
Component	backend/server.js:69

Email đã đăng ký ⇒ `200` ; email chưa đăng ký ⇒ `404 {"error":"User not found"}` .

Kẻ tấn công dò được danh sách email có tài khoản thật. **Đề xuất:** luôn trả `200` với thông báo trung tính.

BUG-03-06 — Backend không kiểm tra độ mạnh mật khẩu

Feature	FR-03 · Severity / Priority High / P1
Test case	FR03-TC10 , FR03-TC10a , FR03-TC10b
Component	backend/server.js:84-95

`POST /api/reset-password` với `newPassword: "1"` ⇒ `200 Password reset successfully` , và đăng nhập bằng `"1"` thành công.

Thậm chí `newPassword: ""` (chuỗi rỗng) cũng được chấp nhận.

Lớp kiểm tra duy nhất nằm ở FE (và chính lớp đó lại sai — BUG-03-01), bỏ qua hoàn toàn khi gọi API trực tiếp.

BUG-03-07 — Mật khẩu lưu plaintext và trả về qua API

Feature	FR-03 · Severity / Priority Critical / P0 · Test case FR03-TC11
Component	backend/database.js:88 · server.js:23, server.js:46

GET `/api/users/me` trả về nguyên vẹn trường `password` đúng bằng chuỗi vừa đặt.

Mật khẩu được so sánh bằng `user.password === password` — không hash, không salt.

Đề xuất: `bcrypt` / `argon2`, và loại bỏ cột `password` khỏi mọi response.

BUG-03-08 — Đặt lại mật khẩu thành công nhưng tài khoản vẫn bị khoá

Feature	FR-03 · Severity / Priority High / P1 · Test case FR03-TC14
Component	backend/server.js:86 (câu UPDATE khi reset)
Evidence	evidence/bugs/BUG-03-08.png

Steps: đăng nhập sai 2 lần (mỗi lần `login_attempts += 2` ⇒ vượt ngưỡng 3, khoá 180 giây) → đặt lại mật khẩu thành công → đăng nhập bằng mật khẩu **mới**.

Expected: khôi phục mật khẩu là hành động xác thực chủ sở hữu ⇒ phải mở khoá tài khoản.

Actual (log thật):

```
POST /api/reset-password      ⇒ 200 {"message":"Password reset successfully"}
POST /api/login (mật khẩu MỚI) ⇒ 403 {"error":"Tài khoản đã bị khóa. Vui lòng thử lại sau."}
```

UI hiện "*Đăng nhập thất bại. Vui lòng kiểm tra lại.*" — không nói tài khoản đang bị khoá,

khiến người dùng tưởng mình đổi mật khẩu hỏng và tiếp tục thử ⇒ càng khoá lâu.

Nguyên nhân: câu UPDATE chỉ đặt `password` và `reset_token`, **không** reset `login_attempts` / `locked_until`.

BUG-08-01 — Khách hàng tự sửa tổng tiền thanh toán ngay trên giao diện

Feature	FR-08
Severity / Priority	Critical / P0
Component	frontend-web/src/pages/Checkout.jsx:96-106 · backend/server.js:302-317
Test case	FR08-TC04
Evidence	evidence/bugs/BUG-08-01.png (ô tiền = 1), BUG-08-01-success.png

Precondition: đã đăng nhập, giỏ hàng chứa iPhone 15 Pro Max — **30.000.000 đ**.

Steps:

1. Vào /checkout .
2. Ô **"Tổng tiền thanh toán (VND)"** là `<input type="number">` sửa được → gõ 1 .
3. Bấm **Xác Nhận Thanh Toán**.

Expected: tổng tiền do **server** tính từ giỏ hàng; mọi giá trị do client gửi phải bị bỏ qua hoặc đối chiếu.

Actual (log thật):

```
Tổng tiền FE tính: 30000000 đ
Đơn hàng trong CSDL: {"id":118,"user_id":724,"total_amount":1,"status":"pending",
"shipping_address":null,"created_at":"2026-08-22 13:56:12"}
```

Màn hình vẫn báo **"Thanh toán thành công!"**.

Tác động: mua hàng 30 triệu với giá 1 đồng. Đây là lỗ hổng gây thiệt hại tài chính trực tiếp.

Đề xuất: bỏ hần ô nhập tổng tiền; backend tính lại từ `items` + giá trong CSDL, từ chối nếu lệch.

BUG-08-02 — /api/checkout chấp nhận tổng tiền âm và không phải số

Feature	FR-08 · Severity / Priority High / P1 · Test case FR08-TC05 , FR08-TC06
Component	backend/server.js:305

- `{"total_amount": -1000000}` ⇒ 200 , đơn hàng lưu tổng **âm** (báo cáo doanh thu sai lệch).
- `{"total_amount": "abc"}` ⇒ 200 , cột khai báo `INTEGER` nhưng SQLite lưu nguyên chuỗi.

Không hề có bước validate kiểu / dấu / khoảng giá trị.

BUG-08-03 — Tạo được đơn hàng từ giỏ hàng rỗng

Feature	FR-08 · Severity / Priority Medium / P2 · Test case FR08-TC07
Component	backend/server.js:302-317

GET /api/cart trả [] nhưng POST /api/checkout {"total_amount": 0} vẫn 200 và sinh orderId.

Backend **bỏ qua hoàn toàn** trường items mà FE gửi lên — không kiểm tra giỏ, không kiểm tra sản phẩm có thật.

BUG-08-04 — IDOR: ai cũng đọc được đơn hàng của người khác

Feature	FR-08
Severity / Priority	Critical / P0
Component	backend/server.js:340 — route GET /api/orders/:id thiếu authenticateToken
Test case	FR08-TC08
Evidence	evidence/bugs/BUG-08-04.png

Steps: user A tạo đơn → **mở trình duyệt sạch, không đăng nhập** → truy cập GET /api/orders/<id của A>.

Actual (log thật):

```
Nạn nhân (user_id=725) tạo đơn #119
GET /api/orders/119 KHÔNG token => 200 {"id":119,"user_id":725,"total_amount":777777,
  "status":"pending","shipping_address":"Địa chỉ riêng tư của nạn nhân",...}
```

Tác động: id đơn hàng là số tự tăng ⇒ duyệt 1..N là lấy được **toàn bộ** đơn hàng và địa chỉ giao hàng của mọi khách.

Đây là rò rỉ dữ liệu cá nhân quy mô lớn.

Đề xuất: thêm authenticateToken và điều kiện WHERE user_id = req.user.id (trừ vai trò admin).

BUG-08-05 — Giỏ hàng không được xóa sau khi thanh toán

Feature	FR-08 · Severity / Priority High / P1 · Test case FR08-TC09
Component	frontend-web/src/pages/Checkout.jsx:8

clearCart được import từ useCart() nhưng **không bao giờ được gọi**. Sau khi thanh toán thành công, quay lại /cart (bằng link trong SPA) thì giỏ vẫn nguyên → bấm thanh toán lần nữa tạo **đơn thứ 2**.

Test đã chứng minh: 1 giỏ hàng ⇒ 2 đơn hàng trong CSDL.

BUG-08-06 — Đơn hàng luôn được lưu với địa chỉ giao hàng rỗng

Feature	FR-08 · Severity / Priority High / P1 · Test case FR08-TC10
Component	frontend-web/src/pages/Checkout.jsx:44-48

FE gửi { items, total_amount, coupon_id } — **không có** shipping_address ; còn backend thì bỏ qua items .

Kết quả: mọi đơn đặt qua giao diện đều có shipping_address: null ⇒ **không thể giao hàng**.

Giao diện cũng không hề có ô nhập địa chỉ.

BUG-08-07 — Mã giảm giá phần trăm làm TĂNG tổng tiền gấp 10 lần

Feature	FR-08
Severity / Priority	Critical / P0
Component	backend/server.js:432-434
Test case	FR08-TC11 , FR08-TC11a , FR08-TC11b
Evidence	evidence/bugs/BUG-08-07.png

Steps: giỏ hàng 30.000.000 đ → nhập SAVE10 (percent, discount_value = 10) → **Áp dụng**.

Expected: giảm 10% ⇒ discount_amount = 3.000.000 , final_amount = 27.000.000 đ .

Actual (log thật):

```
POST /api/apply-coupon ⇒ 200 {"success":true,"coupon_id":1,
  "discount_amount":-270000000,"final_amount":300000000,
  "message":"Áp dụng thành công! Giảm 10%"}
```

Giao diện hiển thị "**Thành tiền: 300.000.000 đ**" kèm dòng chữ xanh "✅ *Áp dụng thành công! Giảm 10%*".

Nguyên nhân:

```
discount_amount = Math.floor(total_amount * (1 - coupon.discount_value));
// với discount_value = 10 ⇒ total * (1 - 10) = -9 * total ⇒ discount Âm
// final = total - (-9*total) = 10 * total
```

Công thức lẫn lộn giữa "tỉ lệ giảm" và "hệ số còn lại", đồng thời quên chia 100.

Đề xuất: discount_amount = Math.floor(total_amount * coupon.discount_value / 100) .

BUG-08-08 — Đơn hàng đúng bằng giá trị tối thiểu bị từ chối (off-by-

one)

Feature	FR-08 · Severity / Priority Medium / P2 · Test case FR08-TC15b
Component	backend/server.js:421

BIGBUY yêu cầu đơn tối thiểu **500.000 đ**. Kiểm chứng 3 mốc:

Tổng đơn	Kết quả thật	Đúng/Sai
499.999 đ	400 chưa đủ giá trị tối thiểu	✓ đúng
500.000 đ	400 chưa đủ giá trị tối thiểu	✗ SAI — đúng bằng mức tối thiểu thì phải được áp
500.001 đ	200, final_amount = 450.001	✓ đúng

Nguyên nhân: `if (total_amount > coupon.min_order_amount)` dùng `>` thay vì `>=`.

Thông báo lỗi nói "*chưa đủ giá trị tối thiểu 500.000 đ*" trong khi đơn **đúng bằng** 500.000 đ ⇒ gây tranh cãi với khách hàng.

BUG-08-09 — Giỏ hàng biến mất khi tải lại trang

Feature	FR-08 · Severity / Priority Medium / P2 · Test case FR08-TC17
Component	frontend-web/src/context/CartContext.jsx:7

`useState([])` thuần, không đồng bộ `localStorage`, không đồng bộ `POST /api/cart`.

Nhấn F5 ở trang giỏ hàng ⇒ hiện ngay "*Giỏ hàng của bạn đang trống*".

Backend **có** API giỏ hàng nhưng FE không dùng — và bản thân `userCarts` cũng chỉ là biến in-memory (`server.js:14`),

mất sạch mỗi lần khởi động lại backend.

BUG-15-01 — Sửa 1 sản phẩm làm toàn bộ bảng đổi sang cùng một tên

Feature	FR-15
Severity / Priority	Critical / P0
Component	frontend-admin/src/App.jsx:107-112
Test case	FR15-TC05
Evidence	evidence/bugs/BUG-15-01-before.png (trước), BUG-15-01.png (sau)

Precondition: đăng nhập admin, bảng có ≥ 3 sản phẩm với tên khác nhau.

Steps: bấm **Sửa** ở đúng 1 sản phẩm → đổi tên → **Lưu sản phẩm**.

Expected: chỉ dòng đó đổi tên.

Actual (log thật):

UI: 6 dòng mang tên mới · CSDL: 1 bản ghi ⇒ lỗi nằm ở FRONTEND

Ảnh `BUG-15-01.png` cho thấy **cả 6 dòng** đều mang tên `EVIDENCE-...-DOI-TEN`, trong khi cột **Giá** vẫn khác nhau

(30.000.000 / 28.000.000 / 45.000.000 / 6.000.000 / 4.000.000 đ) — bằng chứng rõ ràng là dữ liệu thật không hề bị đổi.

Nguyên nhân:

```
await axios.put(`${API_URL}/products/${productForm.id}`, productForm);
const fakeMassUpdatedProducts = products.map((p) => ({ ...p, name: productForm.name }));
setProducts(fakeMassUpdatedProducts); // gán tên của SP đang sửa cho TẤT CẢ sản phẩm
```

Tác động: admin nhìn thấy dữ liệu **hoàn toàn sai** cho tới khi reload. Rủi ro thao tác nhầm rất cao:

admin có thể bấm **Xóa** nhầm sản phẩm vì mọi dòng trông giống hệt nhau.

Đề xuất: thay 3 dòng trên bằng `fetchData()` như nhánh thêm mới.

BUG-15-02 — Thêm/sửa/xoá sản phẩm không cần đăng nhập

Feature	FR-15
Severity / Priority	Critical / P0
Component	<code>backend/server.js:161</code> , <code>:173</code> , <code>:186</code> — cả 3 route đều thiếu <code>authenticateToken</code>
Test case	<code>FR15-TC07</code> , <code>FR15-TC08</code> , <code>FR15-TC09</code>
Evidence	<code>evidence/bugs/BUG-15-02.png</code>

Steps: từ một HTTP client **không gửi header** `Authorization`, gọi lần lượt POST / PUT / DELETE `/api/products`.

Actual (log thật):

```
POST /api/products      KHÔNG token => 200 {"message":"Product created","id":250}
PUT /api/products/250   KHÔNG token => 200 {"message":"Product updated"}
DELETE /api/products/250 KHÔNG token => 200 {"message":"Product deleted"}
```

Tác động: bất kỳ ai trên Internet cũng **xoá sạch được toàn bộ catalogue** bằng một vòng lặp `for`.

Đối chiếu: `/api/categories` và `/api/admin/*` **có** `authenticateToken` $\Rightarrow$ đây là thiếu sót bị bỏ quên, không phải chủ ý.

Đề xuất: thêm `authenticateToken` + kiểm tra `req.user.role === 'admin'` cho cả 3 route.

BUG-15-03 — Người dùng thường tạo được sản phẩm (không phân quyền theo role)

Feature	FR-15 · Severity / Priority Critical / P0 · Test case FR15-TC10
---------	---

Token hợp lệ với `role = "user"` gọi `POST /api/products` $\Rightarrow$ `200 Product created`.

Trong toàn bộ `server.js`, `req.user.role` **không được kiểm tra ở bất kỳ đâu** — kể cả các route `/api/admin/*`

(chúng chỉ kiểm tra "có token hợp lệ", không kiểm tra vai trò).

Giao diện admin chặn `role !== 'admin'` (`App.jsx:64`) nhưng đó chỉ là rào chắn phía client, bỏ qua được hoàn toàn.

BUG-15-04 — Chấp nhận sản phẩm có tên rỗng / thiếu tên / chỉ khoảng trắng

Feature	FR-15 · Severity / Priority Medium / P2
Test case	FR15-TC11a, FR15-TC11b, FR15-TC11c

Dữ liệu gửi lên	Kết quả	Giá trị trong CSDL
<code>name: ""</code>	200	<code>""</code>
không gửi <code>name</code>	200	<code>null</code>
<code>name: " "</code>	200	<code>" "</code>

Sản phẩm không tên vẫn hiển thị trên cả trang bán hàng lẫn trang admin (dòng trống, không bấm được để phân biệt).

Lớp chặn duy nhất là thuộc tính `required` của HTML5 trên form admin (`App.jsx:496`) — xem `FR15-TC20`.

BUG-15-05 — Chấp nhận giá âm và giá bằng chữ

Feature	FR-15 · Severity / Priority High / P1
Test case	FR15-TC12a , FR15-TC12b , FR15-TC13

`price: -5000` ⇒ lưu `-5000`; `price: "abc"` ⇒ lưu nguyên chuỗi `"abc"` dù cột khai báo `INTEGER`.

Sản phẩm giá âm vào giỏ hàng sẽ làm **giảm** tổng tiền — kết hợp BUG-08-01/BUG-08-07 thành chuỗi khai thác tài chính.

Trang chủ hiển thị `Number("abc").toLocaleString()` ⇒ `NaN VND`.

BUG-15-06 — Giá vượt `MAX_SAFE_INTEGER` bị sai lệch giá trị

Feature	FR-15 · Severity / Priority Medium / P2 · Test case FR15-TC14a , FR15-TC14b	
Giá gửi lên	Giá đọc lại	
<code>9007199254740991</code> (<code>MAX_SAFE_INTEGER</code>)	giữ nguyên	✓
<code>9007199254740993</code> (vượt ngưỡng)	khác giá trị đã gửi	✗ mất chính xác số nguyên

Không có kiểm tra khoảng giá trị hợp lệ ⇒ dữ liệu tiền tệ bị hỏng âm thầm, không báo lỗi.

BUG-15-07 — Xóa sản phẩm không tồn tại vẫn báo thành công

Feature	FR-15 · Severity / Priority Medium / P2 · Test case FR15-TC15
Component	<code>backend/server.js:186-191</code>

```
DELETE /api/products/99999999 ⇒ 200 {"message":"Product deleted"}
```

Handler không kiểm tra `this.changes === 0`. Client không phân biệt được "đã xóa thật" với "không có gì để xóa"

⇒ che giấu lỗi đồng bộ dữ liệu.

BUG-15-08 — `GET /api/products/:id` trả `200 {}` thay vì `404`

Feature	FR-15 · Severity / Priority Medium / P2 · Test case FR15-TC16
Component	<code>backend/server.js:157</code>

`if (!row) return res.status(200).json({})` — sai chuẩn REST.

Hậu quả thấy được ở FE: `ProductDetail.jsx:33` phải xử lý riêng bằng thông báo

"Sản phẩm không tồn tại (Lỗi trắng trang do data rỗng)" — chính lập trình viên SUT cũng thừa nhận trong comment.

BUG-15-09 — Sản phẩm id chẵn trả `price` kiểu chuỗi, id lẻ trả kiểu số

Feature	FR-15 · Severity / Priority High / P1 · Test case FR15-TC17a , FR15-TC17b
Component	backend/server.js:158

```
if (row.id % 2 === 0) row.price = row.price.toString();
```

Request	typeof price
GET /api/products/1	number
GET /api/products/2	string

Cùng một endpoint, cùng một trường, hai kiểu dữ liệu khác nhau tùy id chẵn/lẻ.

Mọi phép tính phía client (`price * quantity`) đều có nguy cơ thành nối chuỗi.

GET /api/products (danh sách) thì luôn trả `number` ⇒ càng khó phát hiện.

BUG-15-10 — Chấp nhận `category_id` trở vào danh mục không tồn tại

Feature	FR-15 · Severity / Priority Medium / P2 · Test case FR15-TC18
Component	backend/database.js:62-69

POST /api/products với `category_id: 999999` ⇒ 200, dù GET /api/categories chỉ có id 1, 2, 3.

Bảng `products` khai báo `category_id` INTEGER **không có** FOREIGN KEY, SQLite cũng không bật PRAGMA foreign_keys.

⇒ dữ liệu mờ côi, lọc theo danh mục sẽ bỏ sót sản phẩm.

BUG-15-11 — Payload XSS lưu nguyên vẹn vào CSDL

Feature	FR-15 (cross-feature) · Severity / Priority High / P1 · Test case FR15-TC19
----------------	---

Tên sản phẩm `<img src=x onerror=window.__xss15=1>` được lưu **nguyên văn**, không escape, không lọc.

Bảng admin render bằng `{p.name}` nên React tự escape ⇒ script **không chạy ở màn hình này** (test đã xác nhận `window.__xss15 !== 1`).

Nhưng dữ liệu độc vẫn nằm trong CSDL, và SUT có sẵn 3 chỗ render thô bằng `dangerouslySetInnerHTML` :

Vị trí	Nguồn dữ liệu
<code>frontend-admin/src/App.jsx:801</code>	<code>shipping_address</code> của đơn hàng
<code>frontend-web/src/pages/Home.jsx:64</code>	từ khoá tìm kiếm
<code>frontend-web/src/App.jsx:26</code>	<code>user.name</code> trên header

Chỉ cần một màn hình mới render tên sản phẩm bằng `dangerouslySetInnerHTML` là lỗ hổng stored XSS được kích hoạt.

Đề xuất: làm sạch dữ liệu ở tầng backend khi ghi, không dựa vào việc React tình cờ escape ở tầng hiển thị.

Việc Khái cần làm với bug report này

- [] Đọc lại 28 bug, xác nhận severity phù hợp với ngữ cảnh môn học.
- [] Tạo **GitHub Issue** cho từng bug (ưu tiên 9 bug Critical) và đính kèm ảnh trong `evidence/bugs/` .
- [] Lệnh soạn sẵn cho `gh CLI`: xem `bug-report/gh-issue-commands.sh` .
- [] Dán link Issue vào cột "Liên kết" của `ai/AI_Log.md` .